
Breaking the Space Barrier and its Application to Language Model Inference

Arip Asadulaev
Generated Computers

Abstract

Directed reachability is the problem behind nondeterministic logarithmic space, and Savitch’s theorem, deterministic space $O(\log^2 n)$, has stood as its barrier for half a century. We prove that the *total* version of the problem STCON_{sf} , in which the algorithm must itself verify that no two vertices are joined by more than k walks, lies in ReachUL , LOGDCFL , C=L and SC^2 , and is decided in deterministic space $O(\log^2 M / \log \log M)$, below Savitch’s bound. We then show that these constructions are the control side of language-model inference. The machinery that enforces a grammar on a model’s output, the automaton, the tokenizer, the stack of a recursive schema, has a step graph in which exactly one path joins any two vertices, and one capped traversal of it computes everything a decode loop needs. Built into an engine for Apple’s MLX and measured on one 16 GB laptop with Qwen3.5-2B and 4B, the theorems give a schema-constrained stream that decodes faster than a free one (180 against 138 tokens/s), recursive schemas at 170 tokens/s against 94, a 1,106-state tool grammar in 3 MB instead of 824 MB, a server that survives its sixteenth grammar where token tables die at the fifteenth, and token-identical outputs in 1.8 times fewer weight streams. The engine’s kernels, which are engineering rather than theory, separately give eight agents 3.1 times the stock library’s throughput (271 against 87 tokens/s) and forty LiveCodeBench problems in less than half the wall time; those results are secondary and are reported in the appendix.

1 Introduction

Given a directed graph and two of its vertices, is there a path from one to the other? This is the complete problem of nondeterministic logarithmic space (NL), and the best deterministic algorithm known for it, Savitch’s, needs $O(\log^2 n)$ space. Whether that bound can be lowered is one of the oldest open questions in the field. Apers and Edenhofer [1] recently showed that a quantum computer with logarithmic memory decides reachability on graphs whose pairs are joined by at most k walks, and asked where the language sits classically.

The theorems. We answer the question for the *total* form of the language, in which the input is an arbitrary graph and the algorithm must reject, within the same small memory, every graph that violates the few-path bound. We prove that it lies in four classes at once, ReachUL , LOGDCFL , C=L and SC^2 , and is decided in deterministic space $O(\log^2 M / \log \log M)$, below Savitch’s barrier (Theorem 1). The four memberships need four proofs, because the four machines compute in different ways, but the proofs share one tool: a lemma that lets a search be cut at its total budget and still be complete or have found a violation (Lemma 2). We also prove two facts about the shape of few-path graphs: between any two vertices, almost every vertex is on a forced chain, and the paths factor into independent blocks (Theorems 5 and 6).

The application. A language model writes text one token at a time, each token costs one reading of the whole model from memory forming a *weight stream*. Moreover, the output must follow a

Table 1: **Headline results.** One Apple M2 Pro (16 GB, wired GPU limit 10.7 GB), Qwen3.5-2B and Qwen3.5-4B in 4-bit precision, greedy decoding. *Stock* is mlx-lm in its own process. The upper block follows from the theorems; the lower block comes from the engine’s kernels alone and is secondary. Full tables in Appendices D and E.

Setting	Result used	STCON (ours)	Baseline
<i>From the theorems (Section 4)</i>			
JSON schema, one stream, 2B / 4B	forced chains (Thm. 5)	180 / 82 tps	138 / 63 tps
recursive schema, one stream, 2B	capped stack (Thm. 4)	170 tps	94 tps
1,106-state typed tool grammar	walk kernel (ReachUL)	3.4 MB	824 MB, 1–5 min to build
16 grammars in one server, 4B	walk kernel (ReachUL)	all 16, +1.1 GB	dies at the 15th, +6.9 GB
18 outputs, weight streams, 2B / 4B	block product (Thm. 6)	374 / 484 , exact	672 / 686
<i>From the engine’s kernels alone, secondary (Appendices D and E)</i>			
8 streams, 2B / 4B	batched kernels	576 / 271 tps	188 / 87 tps
LiveCodeBench, 8 streams, 4B	batched kernels	476 s	1,181 s
6 streams thinking 16k tokens, 4B	cache handling	peak 7.2 GB	stock: out of memory

grammar, a JSON schema, a typed tool call, a recursive document, and increasingly several such *agents* run at once. The machinery that enforces the grammar is deterministic, and its graph of configurations has the few-path property in the strongest form: exactly one path joins any two vertices. Theorem 7 makes this precise for the graph of one decoding step, for finite automata and pushdowns alike, and every one of our proofs then becomes a mechanism: the unambiguous traversal computes the grammar’s whole table in one pass. The forced chains are output the engine writes without the model, which is why a constrained stream decodes *faster* than a free one. The recompute-rather-than-store discipline behind the ReachUL proof removes the table altogether (a kernel walks every token through the automaton on the device each step). The capped stack machine is the constraint for recursive sachems. And the block product lets independent parts of an output be decoded in parallel and certified exact. Table 1 gives the results on one Apple silicon laptop in two groups: the results that follow from the theorems, which are the subject of this paper and of Section 4, and the results of the engine’s kernels alone, which are secondary and are reported in Appendices D and E.

Contributions and organization. (1) The four class memberships, the space bound and the hardness result for total few-path reachability, with the truncation lemma that unifies the proofs (Section 2). (2) The structure theorems: branch kernel and block product. (3) The step-graph theorem, which identifies the control side of decoding as a mangrove and reduces every constraint type to one traversal (Section 3). (4) Five mechanisms derived from the proofs, built into an engine, and measured for speed and memory on one laptop (Section 4); the engine’s own throughput results are secondary and are kept in the appendix. The main text is self-contained, appendix A explains the inference terms in plain words, Appendix B holds the remaining proofs and algorithms, Appendix C the mechanisms in detail, and Appendices D and E every measurement.

2 Theory

2.1 Walks, few paths, and the language

Definition 1 (Walks and their number). Let G be a finite directed graph with n vertices. A *walk* from u to v is a sequence of edges that starts at u and ends at v ; vertices may repeat, and the empty walk counts as one walk from every vertex to itself. $N_G(u, v) \in \mathbb{N} \cup \{\infty\}$ is the number of walks from u to v , it is infinite exactly when a directed cycle can be reached from u and can reach v .

Definition 2 (Few paths). For $k \geq 1$, G is *k-strongly few* if $N_G(u, v) \leq k$ for every ordered pair of vertices. A 1-strongly-few graph is a *mangrove*: from any vertex, every vertex it can reach, it reaches in exactly one way. Because the empty walk is counted, a k -strongly-few graph has no cycles.

The language is

$$\text{STCON}_{\text{sf}} = \{ \langle G, s, t, 1^k \rangle : N_G(u, v) \leq k \text{ for all } u, v, \text{ and } N_G(s, t) \geq 1 \},$$

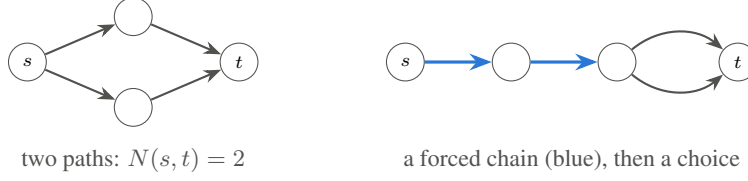


Figure 1: **Left**, the smallest graph with two walks from s to t , it is 2-strongly few. **Right**, a chain of vertices with one way in and one way out is *forced*. Theorem 5 says that in a k -strongly-few graph, once forced chains are contracted, at most $2k$ vertices remain between any two vertices, in decoding, a forced chain is output the model does not need to be asked about.

with k in unary and G by its adjacency matrix; M is the input length, so $n \leq M$ and $nk \leq M^2$.

In a *mangrove* there is never a choice of route: if you know where you started and where you are, you know the way you came. Fig. 1 shows the smallest graph that is not a mangrove and the shape that matters most later, a *forced chain*, a run of vertices with one way in and one way out, which whoever enters must walk to its end. The word *and* in the language is the whole difficulty: the input may be any graph, with cycles or exponentially many paths, and the algorithm must detect that within the same memory it uses for good graphs. This is what separates the total language from the *promise* problems usually studied, in which the algorithm may assume the graph is well behaved.

2.2 The Main Theorem

Theorem 1 ([2]). $\text{STCON}_{\text{sf}} \in \text{ReachUL} \cap \text{LOGDCFL} \cap \text{C=L} \cap \text{SC}^2$, and $\text{STCON}_{\text{sf}} \in \text{DSPACE}(O(\log^2 M / \log \log M))$. The language is StrongFewL-hard under deterministic logspace many-one reductions.

Four different kinds of small-memory machine each decide whether a graph has few paths and whether t is reachable (Fig. 2): an unambiguous nondeterministic machine, for which no configuration is reachable by more than one computation (ReachUL [5]), a deterministic machine with a stack, in polynomial time (LOGDCFL [3]), a machine that tests whether a signed count is zero (C=L), and one deterministic machine that is fast *and* small at the same time, polynomial time with $O(\log^2 M)$ space (SC^2). The space bound is the barrier result: general reachability is not known to be decidable in less than Savitch's $\log^2$ space, and here it is, by way of the unambiguous machine and the simulation of Allender and Lange [4]. This theorem matter because it is a total language, so every algorithm in it must *verify* the few-path property, and that verification is what makes the constructions safe to reuse, a search that is complete or has found a violation. Each proof is a different construction.

2.3 How the proofs go

All four proofs bound a search by a budget, and one lemma says why that is allowed.

Lemma 2 (Budget truncation). *Let H be a finite directed graph with a root r , give every vertex v a budget $\kappa(v) \in \mathbb{N}$, and let $B = \sum_v \kappa(v)$. Write $N^{\leq B}(r, v)$ for the number of walks from r to v of length at most B . Then*

$$N_H(r, v) \leq \kappa(v) \text{ for all } v \iff N_H^{\leq B}(r, v) \leq \kappa(v) \text{ for all } v.$$

Proof. Left to right is immediate, since $N^{\leq B} \leq N$. For the converse, suppose the bounded counts respect the budgets and some walk from r has length at least B . Its first $B + 1$ prefixes, of lengths $0, \dots, B$, are $B + 1$ distinct walks from r of length at most B , and each ends at some vertex; since the budgets sum to $B < B + 1$, some vertex v is the endpoint of more than $\kappa(v)$ of them, a contradiction. So every walk from r is shorter than B , and $N_H = N_H^{\leq B}$. $\square$

Explore walks only up to a length equal to the total budget: either you have seen everything, or you have already found a violation. There is no third case. We say such a search *fails closed*. Every cap in the engine of Section 3 has this form.

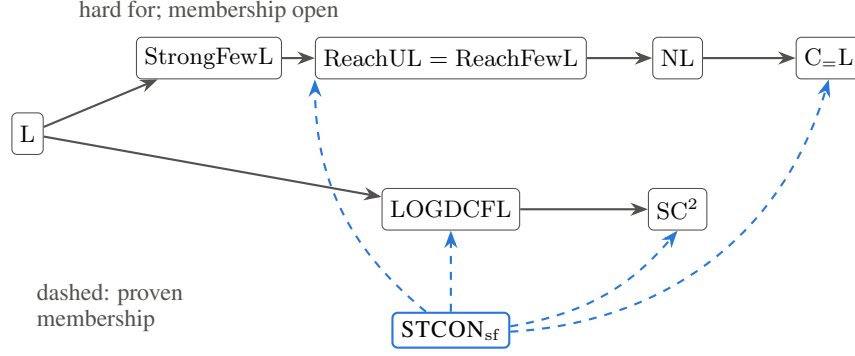


Figure 2: **Where the language lands.** Solid arrows are known inclusions, the upper row bounds the ambiguity of a nondeterministic logspace machine, the lower row gives a deterministic machine a stack. Whether NL is contained in LOGDCFL or in SC^2 is open, which is why the four memberships (dashed) needed four proofs. Typical problems: undirected reachability for L, directed reachability for NL, reachability in a rooted mangrove for ReachUL, a deterministic context-free language for LOGDCFL, singularity of an integer matrix for $C=L$.

Construction 3 (Universal source). Add a vertex u^* with an edge to every vertex of G . Then $N(u^*, v) = \sum_a N(a, v)$, so a bound on all pairs becomes one bound from one root, with $\kappa(v) = \sum_a \kappa(a, v)$, and no count between old vertices changes.

Route 1: a deterministic machine with a capped stack (LOGDCFL). Walk the tree of all walks from a source in depth-first order, keeping only a stack of return points (a vertex and the index of its next successor, $O(\log M)$ bits per frame), and abort as soon as more than nk nodes have been entered (Algorithm 1 in Appendix B).

Theorem 4 (Capped stack traversal). *If G is k -strongly few, the traversal from every source completes within nk nodes and its counts are exact, if G has a directed cycle, some source overflows. The decider that runs the traversal from every source, rejecting on overflow, then recounts walks to every target, rejecting when a count exceeds k , and accepts iff the (s, t) count is positive, accepts exactly $STCON_{sf}$, it is a deterministic auxiliary pushdown automaton with an $O(\log M)$ work tape running in polynomial time, so $STCON_{sf} \in LOGDCFL \subseteq SC^2$.*

Proof sketch. In a k -strongly-few graph each of the n vertices ends at most k walks from any source, so the tree of walks has at most nk nodes and every traversal completes, the graph is acyclic, so every walk is a simple path and the truncation loses nothing. If there is a cycle, start on it: following it gives a distinct walk of every length $0, \dots, nk$, so the traversal enters $nk + 1$ nodes and overflows. The order of the passes matters: a target count starts only after every source has bounded the whole tree, so a bad input cannot force an exponentially long recount before its violation is detected. Cook's characterization of LOGDCFL by polynomial-time deterministic AuxPDAs with a logarithmic work tape gives the membership, and his simulation places LOGDCFL in SC^2 [3]. $\square$

Route 2: an unambiguous traversal (ReachUL). Walks from a root are hashed into a *layered* graph, one layer per step up to the total budget B of Lemma 2, by a polynomial fingerprint of the walk evaluated at a point of a finite field. If no two walks collide, the layered graph is reach-unambiguous from its root and the number of its states in the fiber of v is $N^{\leq B}(r, v)$. Reach-unambiguity of a rooted graph is itself a total language in ReachUL [5], so the recognizer *certifies* unambiguity before it counts, a separating point exists for every valid input, and on an invalid input either the certificate fails or a count exceeds its budget. *In words:* never visit a configuration twice, because there is one way to reach it; this is the discipline that the engine turns into a table computed in one pass and, in its purest form, into a table that is never stored at all.

Route 3: an exact count ($C=L$). The bounded counts $N^{\leq B}(r, v)$ are $\#L$ functions (paths in a layered graph). For each v the product $\prod_{j=0}^{\kappa(v)} (N^{\leq B}(r, v) - j)$ vanishes exactly when the count is within its budget, the sum over v of the squares of these products, plus a term for reachability of t , is a GapL function that is zero iff the input is in the language.

The space bound and hardness. Allender and Lange simulate reach-unambiguous logspace machines in $O(\log^2 M / \log \log M)$ deterministic space [4], which with Route 2 gives the space bound, it carries no time claim, and no collapse of classes follows. For hardness, the configuration graph of a StrongFewL machine is strongly few with a polynomial bound and reachability of its accepting configuration is the language, which is the reduction.

2.4 The shape of few-path graphs

Throughout, restrict a graph to the vertices and edges on some s - t path, the result is a directed acyclic graph H with $r = N_H(s, t)$ paths, and $c(H) = |E| - |V| + 1$ is its number of independent cycles when directions are ignored.

Theorem 5 (Branch kernel). $c(H) + 1 \leq r \leq 2^{c(H)}$. Contracting every internal vertex with one way in and one way out leaves a multigraph K with the same number of s - t paths, $|V(K)| \leq 2c + 2 \leq 2r$ and $|E(K)| \leq 3c + 1 \leq 3r - 2$. Hence between any two vertices of a k -strongly-few graph, the subgraph of vertices on paths between them is a subdivision of a multigraph with at most $2k$ vertices and $3k - 2$ edges.

Proof sketch. The unit s - t flows form a polytope of dimension c whose vertices are exactly the path indicators, and a c -dimensional polytope has at least $c + 1$ vertices; fixing one path P_0 , the map $P \mapsto \chi_P \oplus \chi_{P_0}$ into the binary cycle space (dimension c) is injective, so $r \leq 2^c$. Summing $d^+(x) - 1$ over the internal vertices gives $|E| - (|V| - 1) = c$, so at most c vertices have two or more ways out, and likewise at most c have two or more ways in; every vertex that survives the contraction is one of these, and contracting a one-in one-out vertex preserves paths and c . $\square$

Theorem 6 (Block product). Let $B_1, \dots, B_q$ be the biconnected blocks on the route from s to t in the block-cut tree of H 's underlying undirected graph, with entry and exit cut vertices a_i, b_i . Then $N_H(s, t) = \prod_i N_{B_i}(a_i, b_i)$ and $c(H) = \sum_i c(B_i)$, if $N_H(s, t) \leq k$, at most $\lfloor \log_2 k \rfloor$ blocks have more than one path.

Proof sketch. A simple s - t path follows the unique block route and cannot enter an off-route block and return, because it would repeat the cut vertex through which it entered, inside each route block it chooses one a_i - b_i path, and arbitrary choices concatenate uniquely. $\square$

On the way between any two vertices, almost everything is forced, and the little that is not factors into independent pieces at the cut vertices. When $k = 1$ there is no choice at all. Both facts are read directly off the graph of a decoding step below: forced chains are free output, and blocks are the parts of an output that can be produced in parallel.

Takeaway. What the theorems hand to the engine. (1) A search stopped at its budget is either complete or has found an error (Lemma 2), so every cap can be trusted. (2) In a one-path graph nothing needs to be visited twice or stored (the ReachUL proof), so a table can be built in one sweep or not built at all. (3) A machine with a capped stack recognizes the language (Theorem 4), so recursion is a stack. (4) Between two vertices almost everything is forced and the rest splits into independent blocks (Theorems 5 and 6), so forced text is free and independent parts can be produced in parallel.

3 Application: the control side of inference is a mangrove

3.1 What a language model does at every step, and why it is slow on a laptop

A language model writes one token at a time. To choose the next token it runs its whole network once: the id of the last token goes in, a score (a logit) for each of the 248,320 tokens of the vocabulary comes out, and the token with the highest score is taken, or one is drawn at random from the scores. We call this one *step*. On a laptop a step is slow for one reason: all the weights must be read from memory at every step, and the model is 1.1 GB (2B) or 2.26 GB (4B) even at 4 bits. Reading them takes 7 or 15 ms; the arithmetic is nearly free next to it. We call one such reading a *weight stream*. Two things follow. If the engine can write a token without running the model, it saves a whole weight stream. And a step that handles several tokens, or several answers at once, costs barely more than a step for one, because the weights are read once either way.

A *constraint* (a JSON schema, the format of a tool call) is a rule about which outputs are acceptable. It is compiled into a small machine, a DFA, that reads the output one byte at a time and is always in one of a few hundred states; at each state some bytes are allowed and the others are not. To enforce the constraint the engine does two things at every step: before the choice it forbids every token whose bytes the deterministic finite automaton (DFA) would reject, by setting their scores to $-\infty$ (the *mask*); after the choice it moves the DFA along the bytes of the chosen token. Existing engines keep, per grammar, a table of DFA states $\times$ vocabulary [6, 7], or compute the mask on the CPU at every step [8]; Appendix A lists what that costs as the agents multiply. The rest of this section is about doing these two things without slowing the model and without spending memory.

Takeaway. A token costs one full read of the weights. The only way to be faster is to skip steps, and the only way to serve many agents is to share steps. A constraint is a small machine that must answer, at every step, which tokens are allowed and where it goes next.

3.2 Why the constraint has one path

Take any DFA state and any token. The token is a fixed string of bytes and the DFA is deterministic, so following those bytes from that state leads to exactly one next state, or to a dead end if a byte is forbidden. There is nothing to search and nothing to guess. This is the whole reason the theory of Section 2 applies: the graph whose vertices are (bytes read so far, DFA state) and whose edges add one byte is a tree, a mangrove with $k = 1$, so every fact about it can be computed once and looked up afterwards. Formally, the control machine is the DFA (for a recursive schema, a DFA with a stack), $\delta(c, b)$ is the state after byte b and $\delta^*(c, w)$ its extension to strings; the vocabulary is stored as a *trie*, the byte strings of all tokens sharing their prefixes, 0.59 million nodes; and the *step graph* at c pairs each trie node with the state at which the DFA reaches it:

$$P_c = \{(w, \delta^*(c, w)) : w \text{ a trie node, } \delta^*(c, w) \text{ defined}\}, \quad (w, c') \rightarrow (wb, \delta(c', b)),$$

rooted at (ε, c) (Fig. 3).

Theorem 7 (The step graph). *For every deterministic control machine and every configuration c :*

1. P_c is an out-tree, so the configuration $T[t, c] = \delta^*(c, \text{bytes}(t))$ reached by a token t is unique, and P_c is a mangrove.
2. Let U join the step graphs of all contexts under a universal source u^* (Construction 3). Then U is a mangrove, and the capped traversal from u^* with budget B either computes $T[t, \kappa]$ for every token and every context or reports that B is exceeded, it never returns a partial table.
3. The vertices of P_c with exactly one way out form the forced continuation of c : the longest byte string every acceptable output from c begins with (the unary chains of Theorem 5).
4. For a forced string, the pre-token boundaries of a byte-pair tokenizer are cut vertices of its tokenization lattice, so the tokenizer’s split is a product of one choice per block (Theorem 6), and only the last block depends on bytes not yet known.

Proof. (1) Every edge lengthens w , so P_c is acyclic, a vertex (wb, c'') has the unique parent $(w, \delta^*(c, w))$, because the trie fixes w and determinism fixes the configuration, so every vertex has one path from the root. (2) The roots of distinct step graphs are distinct vertices, so u^* gives every vertex of U exactly one path, the traversal is Construction 3 on U , and Lemma 2 with unit budgets says that within B steps it has either visited every vertex once or found the budget exceeded. (3) A vertex with one out-edge has one allowed byte, along a run of such vertices every acceptable continuation must take that byte, and the run ends at the first vertex with two allowed bytes or an accepting configuration. (4) A tokenizer that never merges across a pre-token boundary makes every tokenization pass through it, which is the definition of a cut vertex, theorem 6 factors the count and the choice. $\square$

Core: From any grammar state, each token leads to exactly one next state. So “which tokens are allowed” and “where does the DFA go” are both entries of a table, and because every state’s tree is separate, the table for all states can be built in one sweep over the vocabulary.

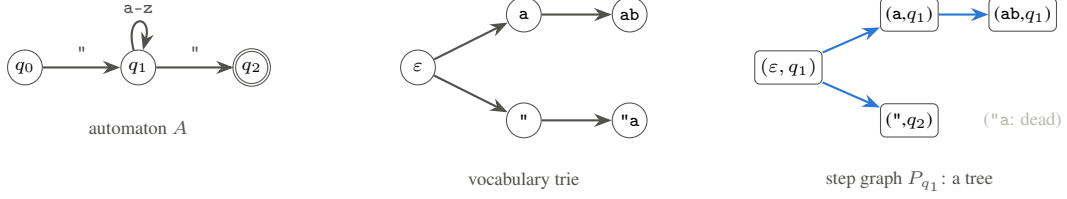


Figure 3: **The step graph.** **Left**, a three-state automaton for a quoted word. **Middle**, part of the vocabulary trie. **Right**, their product from state q_1 : each trie node appears with the state the automaton is in when it gets there, with exactly one route from the root. The token "a is dead from q_1 because after the closing quote no letter is allowed.

3.3 Five mechanisms, and why each one works

Each mechanism is one of the proofs of Section 2 read as a recipe. For each we say what the problem is, what the engine does, why it is correct, and what it measured; Appendix C gives the algorithms.

1. One sweep builds the whole table (Section C.1). *Problem:* the engine needs, for every state and every token, whether the token is allowed and where it leads; a naive build walks every (state, token) pair separately. *What we do:* walk the vocabulary trie once, byte depth by byte depth, carrying all states side by side in one array, so each level is one vectorized lookup. *Why it works:* by Theorem 7(2) the trees of all states joined under one source are still a tree, so one traversal visits every (trie node, state) pair exactly once, and the budget of Lemma 2 makes it stop cleanly if the grammar is too large. *Result:* 0.2–0.6 s per schema; the table is 3 bytes per (state, token), 77 MB for 104 states, which is the memory that mechanism 3 removes.

Core: Build the table with one sweep, not one search per state; the sweep either finishes exact or says the grammar is too large.

2. Text the grammar forces is written for free (Section C.2). *Problem:* much of a JSON output is not chosen by the model at all: after "name": "Maria" the schema allows only , "age":, yet a normal decoder still runs the model once per token for it. *What we do:* after each sampled token the engine looks up the forced continuation of the new state (the bytes along which only one byte is allowed), splits it into tokens the way the tokenizer would, and feeds the sampled token and the forced tokens to the model in one step of $m + 1$ positions. *Why it works:* those states are the one-way-out vertices of Theorem 5, so the text is certain before the model runs; a step of $m + 1$ tokens costs about one weight stream; and the model's caches advance exactly as if it had sampled the tokens one by one, so the output is identical. *Result:* about 30% of the tokens of a JSON output are forced; a constrained stream runs at 180 tokens/s against 138 free on the 2B and 82 against 63 on the 4B.

Core: Certain text costs nothing: it is fed in the same step as the sampled token, which is why a constrained stream is faster than a free one.

3. No table at all (Section C.4). *Problem:* the table grows with the grammar (1.5 GB at 1,970 states), is kept for the life of the server, and every new schema adds another. *What we do:* keep only the DFA (512 bytes per state) and the vocabulary's bytes, and at every step let a GPU kernel recompute the one row the step needs: one thread per token walks the token's bytes through the DFA from the current state and writes the state it reaches, or "dead". *Why it works:* the row is exactly the table's column, because every token has one path (Theorem 7(1)), so nothing is lost by not storing it; and the kernel takes 7–18 μ s against a 7–15 ms step, so nothing is lost in time either. It is the ReachUL proof's discipline, recompute rather than store, made literal. *Result:* 2.9–3.85 MB per grammar against 77 MB–1.5 GB, identical outputs; a 1,106-state typed tool-call grammar costs 3.4 MB instead of 824 MB; a server holds sixteen grammars in 1.1 GB.

Core: Because every token has one path, the table's row can be recomputed on the GPU each step in microseconds, so the table need not exist.

4. Recursive schemas with a small stack (Section C.5). *Problem:* a schema that refers to itself (a tree of nodes with children) is not a DFA; the leading library unrolls three levels and then breaks. *What we do:* each recursive definition becomes its own DFA; a reference pushes a return state on a small stack and jumps into the callee; the callee’s closing bracket pops. The stack, 64 entries, lives on the GPU next to the state. *Why it works:* this is the capped stack machine of Theorem 4; determinism is checked at compile time and the depth cap fails closed. *Result:* 170 tokens/s on the 2B against 143 and 94 for llguidance with and without fast-forward.

Core: A recursive grammar is a DFA plus a capped stack, the machine of the LOGDCFL proof, and it runs on the GPU like the flat case.

5. Independent parts in parallel, certified (Section C.6). *Problem:* a decoder spends one weight stream per token even where the values of a JSON object do not depend on each other as far as the grammar is concerned. *What we do:* decode the values at the same time, as rows of one batch that share the prompt’s cache; then check the assembled document in one extra step: run the model over it once and ask, at every drafted token, whether the model would have chosen exactly that token; where the answer is no, redo that value from the first wrong token. *Why it works:* the state after each key is a cut vertex (Theorem 6), so the grammar lets each value start on its own; and the check is the very test sequential greedy decoding makes at each position, so what passes is exactly the sequential output. The grammar guarantees the language side, the certificate the model side. *Result:* token-identical output with 374 instead of 672 weight streams (2B) and 484 instead of 686 (4B); on this GPU, whose step is compute-bound from four rows, not yet less wall time.

Core: The grammar says which parts of the output are independent; the model is asked once whether the parallel draft is what it would have written. Fewer weight streams, same output.

3.4 The engine around the mechanisms

The mechanisms sit in a decode loop with three pieces of ordinary systems work (Section C.7): matrix-vector kernels for 4-bit weights tuned for 1–16 rows, so a few agents read each weight once; one fused kernel for the recurrent layer’s step; and a loop that builds the next step while the current one runs, with the DFA state and the mask on the GPU, so the host is never waited for. This is engineering, held fixed on both sides of every constrained-decoding comparison; in the free-decoding comparisons with the stock library it is the whole difference.

Takeaway. Put together: a constraint is not a tax on the model but information the engine reads off a one-path graph. What is certain is free, what is independent is parallel, what is needed each step is recomputed, and what recurses is a stack.

4 Results

This section reports the results that follow from the theorems, each measured against the strongest baseline for it. The results of the engine’s kernels alone are secondary: they are reported in Appendices D and E and summarized in one paragraph at the end.

All measurements are on one Apple M2 Pro (19-core GPU, 16 GB, wired GPU limit 10.7 GB) with Qwen3.5-2B (18 recurrent and 6 attention layers) and Qwen3.5-4B (24 and 8), both 4-bit, greedy unless stated; comparisons between our configurations are made inside one process with interleaved rounds and medians (spread under 2%); stock mlx-lm [10] runs in its own process; llguidance [8] runs as a controller inside our loop with the same kernels, so that difference is the control side alone. Memory is MLX’s own active and peak counters. Every table names its script, appendices D and E hold the full versions.

A constrained stream is faster than a free one (Theorem 5; mechanism 2). On the same prompts a schema-constrained stream decodes at 180 tokens/s where the free stream runs at 138 on the 2B, and at 82 against 63 on the 4B: $1.31\times$ on both models, because about 30% of the output tokens are forced chains and are fed without a weight stream. llguidance’s fast-forward mode feeds a similar share of tokens but computes its mask on the host at every step and lands at the speed of free decoding (136 and 69); ours is $1.2\text{--}1.3\times$ faster than it (Table 6).

Table 2: **Speed and weight streams, from the theorems** (full tables: Tables 6, 7 and 13). Top: one stream, tokens/s of decode on the same prompts; the baseline is free decoding / llguidance’s fast-forward mode for JSON schemas, and llguidance’s fast-forward / plain mode for recursive schemas. Bottom: weight streams for 18 outputs, sequential decoding against certified block-parallel decoding, which produces the same tokens in every case.

one stream, tokens/s	2B		4B	
	STCON	baseline	STCON	baseline
JSON schema (forced chains, Theorem 5)	180	138 / 136	82.3	63.2 / 68.6
recursive schema (capped stack, Theorem 4)	170	143 / 94	78.6	67.4 / —
weight streams, 18 outputs	sequential	blocks	sequential	blocks
certified blocks (block product, Theorem 6)	672	374 , 18/18 exact	686	484 , 18/18 exact

Table 3: **Memory, from the ReachUL walk kernel** (full tables: Tables 14 and 15). Top: one grammar, the token table against the walk kernel, with decode tokens/s on the 4B. Bottom: one server process fed sixteen grammars in turn, prompt cache 50 MB.

grammar	states	table	build	walk	tok/s table / walk
person (JSON schema)	104	77 MB	0.7 s	2.9 MB	75 / 75
typed tool calls	1,106	824 MB	62–127 s	3.4 MB	89 / 87
96 fields	1,970	1.5 GB	not built	3.85 MB	— / 121
sixteen grammars in one server	grammars served		above the model	longest first request	
2B, token tables	15 of 16, then out of memory			8.2 GB	26 s
2B, walk kernel	16 of 16			0.6 GB	2.8 s
4B, token tables	14 of 16, then out of memory			6.9 GB	26 s
4B, walk kernel	16 of 16			1.1 GB	6.6 s

Recursive schemas (Theorem 4; mechanism 4). The pushdown decodes recursive schemas at 170 tokens/s on the 2B, $1.19\times$ llguidance’s fast-forward mode and $1.6\times$ its plain mode, and at 78.6 against 67.4 on the 4B, because forced spans cross the returns of the stack: after a node’s last value the whole closing-and-reopening continuation is one span (Table 7).

Decisions, not tokens (Theorem 6; mechanism 5). Over 18 outputs the sequential decoder needs 672 weight streams on the 2B and the certified block decoder 374 ($1.80\times$ fewer); on the 4B, 686 against 484 ($1.42\times$). All 36 outputs are token-identical to the sequential ones. Wall time is not yet lower on this GPU, whose step is compute-bound from four rows; the count of weight streams is the hardware-independent number (Table 13).

The table need not exist (the ReachUL proof; mechanism 3). A token table is $3|Q|V$ bytes and lives for the process. The walk kernel makes a grammar cost 3 MB whatever its size, at the table’s speed and with identical output, and the typed tool-call grammar that the server could not afford as a table (824 MB, minutes to build) costs 3.4 MB: on eight concurrent agent requests, 46 of 46 tool calls have every argument present and of its declared type on the 2B (Table 9). In a server that sees many grammars, tables kill the process at the fifteenth (4B) or sixteenth (2B) grammar, after first-request waits of 26 s and a collapse of throughput; the walk kernel serves all sixteen within 0.6–1.1 GB (Table 15, Fig. 4).

Secondary: what the engine’s kernels add (Appendices D and E). These results do not follow from the theorems and would hold without them. Our 4-bit kernels read the weights once per step for all rows of a batch, so eight streams decode at 576 tokens/s on the 2B and 271 on the 4B against 188 and 87 for stock mlx-lm, whose batch generator stalls between four and eight streams; mlx-lm’s own loop on our kernels reaches the same numbers, and the same prompt on every stream changes nothing, so the gain is the kernels and fp16, not the loop and not any sharing (Table 8). The same kernels finish forty LiveCodeBench problems in $2.2\text{--}2.5\times$ less wall time (Table 10), and the engine’s cache handling lets six 16k-token thinking streams fit where the stock library runs out of memory (Section E.3).

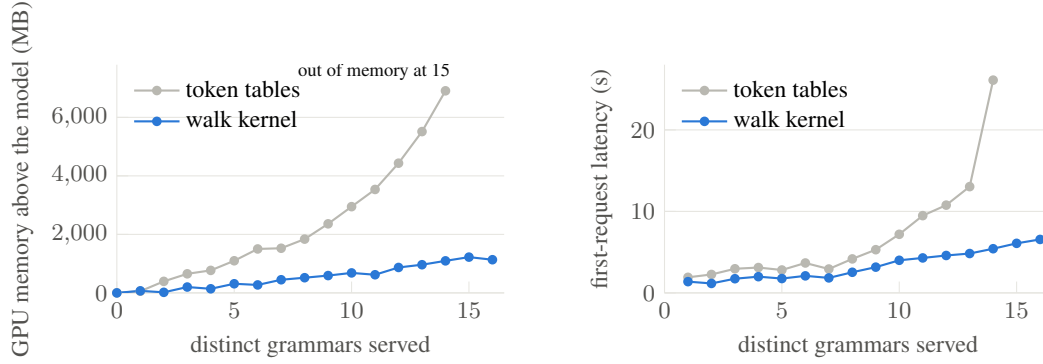


Figure 4: **Constraint memory in the wild, Qwen3.5-4B.** One server process fed sixteen distinct grammars in turn (the six schemas, then schemas of 8 to 96 fields), four requests each, three concurrent, the prompt cache held at 50 MB: active GPU memory above the model, and the latency of each grammar’s first request, which includes its compile. With token tables the fourteenth grammar’s first request waits 26 s and throughput collapses to 11 tokens/s, the fifteenth kills the process. With the walk kernel all sixteen are served within 1.1 GB.

Takeaway. From the theorems: a constrained stream is $1.3\times$ faster than a free one; recursive schemas decode $1.2\text{--}1.6\times$ faster than the best library; a grammar costs 3 MB instead of up to 1.5 GB, so a server survives sixteen grammars; independent parts of an output are decoded in parallel with $1.4\text{--}1.8\times$ fewer weight streams and identical output. From the engine alone, secondary: $3\times$ the stock throughput at eight streams.

5 Discussion and conclusion

The class memberships concern logarithmic memory and change nothing on a machine with gigabytes of it; what transfers is the object each proof constructs and the invariant that makes it correct. The unambiguous traversal exists because a few-path graph can be traversed without revisiting a configuration, the constraint’s step graph has the property in its strongest form, $k = 1$, which is why one pass yields the table for every state, why forced output can be read off the graph, and why the table need not exist. The capped stack machine exists because a deterministic pushdown recognizes the language, applied to a schema’s grammar, it is the recursive constraint. The truncation lemma, which every proof uses, is why every cap in the engine can be trusted. The two constructions are one (Theorem 7): the step graph of a deterministic control machine is a mangrove whether the machine has a stack or not, and a single capped traversal serves both. The cost thesis is what the construction adds to practice: on a memory-bound machine the weight stream is the cost, the theorems cannot change what a stream reads, and they say exactly how many streams an output needs, its decisions.

Two limits are worth stating. The theory applies where inference is a deterministic machine, the dense arithmetic is not one, and ideas that treated it as such (a deferred recurrent state, a certified low-rank head, sibling subtrees decoded in parallel by the block product) were measured and do not pay, which is also why certified blocks save streams and not yet time on this GPU. And the measurements are one machine and one model family, with our own engine’s prefill transient reported as the place where it spends more memory than it should. What the results show is that on a laptop, a constraint need not be a tax on speed nor a table in memory; that one construction suffices for every deterministic constraint, and that the cost of a structured output is its number of decisions, which the few-path graph states exactly, because there is one path.

References

- [1] S. Apers and S. Edenhofer. Quantum logspace and few-path reachability. 2025.
- [2] Total strongly-few directed reachability: unambiguity, auxiliary pushdowns, exact counting, and the StrongFewL boundary. Manuscript, 2026.

- [3] S. A. Cook. Deterministic CFL’s are accepted simultaneously in polynomial time and log squared space. STOC, 1979.
- [4] E. Allender and K.-J. Lange. $\text{RSPACE}(\log n) \subseteq \text{DSPACE}(\log^2 n / \log \log n)$. Theory of Computing Systems, 1998.
- [5] B. Garvin, D. Stolee, R. Tewari, and N. V. Vinodchandran. ReachFewL = ReachUL. Computational Complexity, 2014.
- [6] B. T. Willard and R. Louf. Efficient guided generation for large language models. arXiv:2307.09702, 2023.
- [7] Y. Dong et al. XGrammar: flexible and efficient structured generation engine for large language models. arXiv:2411.15100, 2024.
- [8] M. Moskal et al. llguidance: super-fast structured outputs. <https://github.com/guidance-ai/llguidance>, 2024.
- [9] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: holistic and contamination free evaluation of large language models for code. arXiv:2403.07974, 2024.
- [10] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert. MLX: efficient and flexible machine learning on Apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- [11] S. Yang, J. Kautz, and A. Hatamizadeh. Gated Delta Networks: improving Mamba2 with delta rule. ICLR, 2025.

A Background in plain words, and the problems measured

This appendix defines the inference terms used in the paper and lists the problems, measured on a laptop, that the mechanisms address.

Decoding. The model is given a *prompt*, a list of tokens, and produces a probability for every token in its *vocabulary* (248,320 tokens for the models here) as the next one. The engine picks one, appends it, and asks again. Picking the most probable token every time is *greedy* decoding; drawing at random from the probabilities is *sampling*. Processing the prompt is the *prefill*; producing the answer token by token is the *decode*. Each decode step is one weight stream.

Caches. A transformer keeps, for every token it has seen, a small vector per layer (the *KV cache*) so that it need not re-read the prompt at every step. The models used here are *hybrid*: three of every four layers are recurrent (Gated DeltaNet [11]) and keep a fixed-size state instead of a per-token cache. On Qwen3.5-4B the KV cache costs 32 KB per token per stream and the recurrent state 50 MB per stream.

Constraints. A *constraint* says which outputs are acceptable: a JSON schema, a regular expression, the grammar of tool calls. It is compiled into an *automaton*, a machine with a finite set of states that reads one byte at a time, at each state some bytes are allowed and lead to a next state, the others are forbidden. To enforce the constraint, the engine forbids, at every step, the tokens whose bytes the automaton would reject (a *mask* over the vocabulary), and moves the automaton along the bytes of the chosen token.

Streams and batches. An answer being decoded, with its prompt, its caches and its automaton state, is a *stream*. When several streams are decoded together, the engine runs one step for all of them per weight stream; this is a *batch*, and its size is the number of streams. In an agent system, streams are agents waiting for an answer. The engine’s *slots* are the largest batch it will run; requests beyond it wait.

Where memory goes. Five things occupy the GPU: the weights, the prompt cache, a store of prefills that can be reused, the streams’ caches, which grow with every token, a transient used while a prompt is prefilled, and the constraints’ tables. Appendix E measures each.

The two engines compared. *Stock* means mlx-lm as shipped, in its own process, with its own kernels and its bf16 arithmetic. STCON is our engine on the same weights, with fp16 arithmetic (which on this

Table 4: **Five things that go wrong on a laptop as the agents multiply.** All numbers measured in this paper (Appendices D–E) on one Apple M2 Pro, 16 GB, with Qwen3.5-2B and Qwen3.5-4B in 4-bit precision.

Problem	What we measured	Where it comes from
Throughput stalls with more agents	stock mlx-lm: 87 tokens/s in total at 4 streams and still 87 at 8 (4B); 186 and 188 (2B)	its decode step is not built for several rows, the weights are streamed once per row
Every grammar costs a table	77 MB (104 states) to 1.5 GB (1,970 states), a typed tool grammar 824 MB and 1–5 minutes to build; tables are kept for the life of the process	the table is (vocabulary $\times$ states), and the vocabulary has 248k tokens
A server dies as grammars accumulate	16 distinct schemas in one server: memory above the model reaches 6.9 GB at the 14th (4B), the first request of a new grammar waits 26 s, throughput collapses to 11 tokens/s, the 15th kills the process	tables plus the model no longer fit in the wired GPU limit (10.7 GB)
Enforcing the grammar costs speed	llguidance decodes a schema-constrained stream at the speed of a free one (136 vs 138 tokens/s)	the mask is computed on the host every step, so host and GPU no longer overlap
Long answers do not fit	six agents thinking for 16,000 tokens each: stock mlx-lm runs out of memory; caches cost 32 KB per token per stream and their growth is paid twice	the cache is grown by copying

GPU is closer to a float32 computation than bf16 is, Section D.5), our kernels, and the mechanisms of this paper. For constrained decoding the baseline is llguidance [8], run as a controller inside our own loop with the same kernels, so that the difference is the control side alone.

A.1 What goes wrong when the agents multiply

We measured what happens on a 16 GB Apple M2 Pro with the stock serving library for that platform (mlx-lm [10]) and the leading library for structured output (llguidance [8]). Table 4 lists the problems with the numbers; everything in it is measured in Appendices D and E.

Three of the five problems are about memory, and they compound: the model takes 2.3 GB, a prompt cache 1.5 GB, each grammar hundreds of megabytes, each long answer hundreds more, and the machine has 10.7 GB of GPU memory to give. The other two are about speed, and they also compound: an agent that must wait for its turn behind seven others, at a stalled aggregate rate, with a constraint that costs a quarter of the speed, is an agent that is not useful.

B Remaining algorithms and proof details

This appendix holds the traversal algorithm and the proof details referred to in Section 2.

The capped traversal. Algorithm 1 is the machine of Theorem 4 in global mode, in target mode for b it also counts the nodes ending at b and aborts at $k + 1$. For a source a and length L , the walk-unfolding tree T_a^L has one node per walk of length at most L from a ; distinct walks are distinct nodes even when they end at the same vertex, so $|T_a^L| = \sum_b N^{\leq L}(a, b)$. (i) If $|T_a^{nk}| > nk$ then G is not k -strongly few: more than nk nodes among n endpoints put at least $k + 1$ distinct walks on some b . (ii) If G is k -strongly few then $|T_a^{nk}| \leq nk$ for every a . (iii) If G has a directed cycle, a source on it has $|T_a^{nk}| \geq nk + 1$. (iv) If G is acyclic, $N^{\leq nk}(a, b) = N(a, b)$, since every walk is a simple path of length at most $n - 1 \leq nk$. Resources: the work tape holds a constant number of $O(\log M)$ -bit counters; each stack frame (v, j) is $2\lceil \log_2(n + 2) \rceil$ bits, a traversal enters at most $nk + 1$ nodes and tests at most n successors per node, so with a full $O(M)$ input scan per adjacency test it costs $O(n^2 k M + nk \log M)$, and there are $n + n^2$ traversals.

The universal source in the engine. Instead of adding a vertex, the pass over the vocabulary trie carries all $|Q|$ automaton states as the columns of one array (Algorithm 2), so one traversal computes the product for every root, the work budget is the token-table form of Lemma 2.

Algorithm 1 Capped traversal from a source a with a budget of nk nodes (n vertices, bound k).

```

1: current  $\leftarrow a$ ; next  $\leftarrow 1$ ; depth  $\leftarrow 0$ ; total  $\leftarrow 1$ 
2: loop
3:   if depth =  $nk$  or next >  $n$  then
4:     if stack empty then return (OK, total)
5:   end if
6:   pop( $v, j$ ); current  $\leftarrow v$ ; next  $\leftarrow j$ ; depth  $\leftarrow$  depth - 1; continue
7: end if
8:    $w \leftarrow$  next; next  $\leftarrow$  next + 1
9:   if (current,  $w$ ) is not an edge then continue
10:  end if
11:  push(current, next); current  $\leftarrow w$ ; next  $\leftarrow 1$ ; depth  $\leftarrow$  depth + 1; total  $\leftarrow$  total + 1
12:  if total =  $nk + 1$  then return OVERFLOW
13:  end if
14: end loop

```

Algorithm 2 All-state token table (the universal-source pass). For each trie node at depth d , parent_d gives its parent and byte_d its last byte; tok_d lists the tokens whose string ends at depth d .

```

1:  $S_0 \leftarrow [0, 1, \dots, |Q| - 1]$   $\triangleright$  one row: the root, every state is a root
2:  $\text{alive}_0 \leftarrow \{\text{root}\}$ ;  $W \leftarrow |Q|$ 
3: for  $d = 1, 2, \dots, \text{depth}(\text{trie})$  do
4:   rows  $\leftarrow$  nodes  $w$  at depth  $d$  whose parent is in  $\text{alive}_{d-1}$ 
5:    $S_d[w, \cdot] \leftarrow \delta[S_{d-1}[\text{parent}_d(w), \cdot], \text{byte}_d(w)]$  for all rows (one gather; DEAD stays DEAD)
6:    $W \leftarrow W + |\text{rows}| \cdot |Q|$ ; if  $W > \text{budget}$  then fail (the grammar is too large for the budget)
7:    $T[t, \cdot] \leftarrow S_d[\text{node}(t), \cdot]$  for every  $t \in \text{tok}_d$ 
8:    $\text{alive}_d \leftarrow \{w : S_d[w, \cdot] \text{ not all DEAD}\}$ ; if empty then break
9: end for

```

C The mechanisms in detail

This appendix gives each mechanism of Section 3.3 with its algorithm; Table 5 at the end is the summary.

C.1 One pass, the whole table

For a regular constraint the control machine is its byte-level deterministic automaton $A = (Q, \Sigma, \delta, q_0, F)$, trimmed and minimized, and the context is the state, so Theorem 7(2) is one pass down the trie with $|Q|$ roots. The token table is $T[t, q] = \delta^*(q, \text{bytes}(t))$, or DEAD, and $\text{allowed}[q, t]$ says whether it is not dead, the end-of-sequence token is allowed exactly at accepting states.

Level d of the trie is an integer array with one row per live trie node and one column per automaton state; each level is one gather from the previous one. For a schema with free-text fields almost every trie node stays alive for some state, so the pass touches the trie essentially once: 0.59 million nodes times 104 states is 61 million lookups for the person schema, done level by level in 0.3 s (Fig. 5). The budget is Lemma 2 for the table: the pass either completes, and the table is exact, or reports that the grammar is too large, there is no partial table. On the device the table is 3 bytes per (state, token): 26 MB of masks and 52 MB of transitions for 104 states, and it grows with the states, which is the memory problem that Section C.4 removes.

C.2 Certain output is written for free: forced spans

A state that is not accepting and has exactly one allowed byte is a vertex with one way out, a run of such states is a forced chain (Theorem 5), and its bytes are certain output. After each sampled token, the engine appends the forced chain that follows it *in the same forward pass*. For the model this is exactly as if those tokens had been sampled one by one; for the machine it is one weight stream instead of several. Once the model has written the closing quote of a value in $\{"name": "Maria",$ the only continuation the schema allows is $, "age":$, seven bytes and two tokens, which are fed together with the quote. Fig. 6 shows a whole output with its sampled and forced tokens: on the six JSON schemas used below, about 30% of all output tokens are forced.

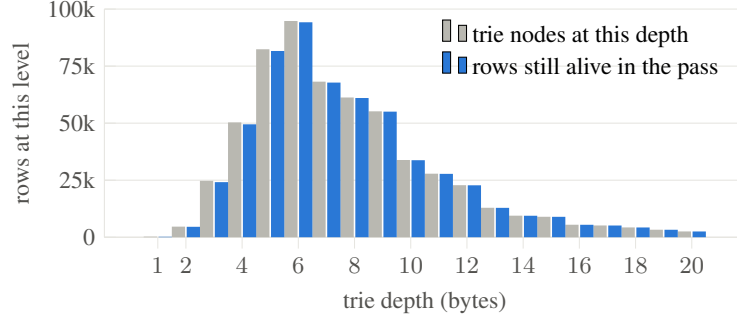
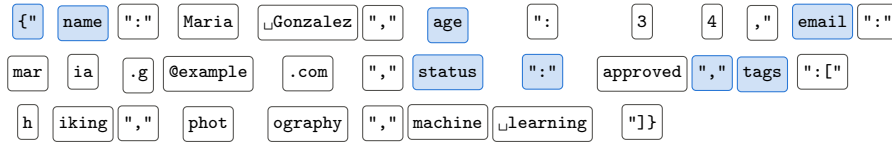


Figure 5: **The all-state pass, level by level.** Trie nodes of Qwen’s 248k-token vocabulary by byte depth (gray) and the rows Algorithm 2 keeps alive at that level for the `person` schema (blue). Because the schema has free-text fields, almost every node is reachable from some state, so the pass touches the trie essentially once, with one (rows $\times$ states) gather per level.



white: sampled by the model (27 tokens, 27 forward passes) blue: forced by the constraint (7 tokens, fed in the same passes)

Figure 6: **One constrained output, token by token.** The 34 tokens of the `person` extraction as the engine produced them on Qwen3.5-2B. Blue tokens are forced spans: after the sampled `"`, `"` the constraint allows only the next key, so `age` is appended in the same forward pass. Every white token cost one weight stream. Note the splits at the keys (`name` then `"`, `:`, not `"` then `:` then `M`), which the boundary rule of Section C.3 enforces.

C.3 Splitting forced text the way the tokenizer would

A forced string of bytes can be split into tokens in many ways, and the model was trained on one of them, the tokenizer’s. Byte-pair tokenizers never merge across the boundaries of the *pre-tokenizer*, which cuts text at words, numbers and punctuation runs, in the lattice of all tokenizations those boundaries are cut vertices (Theorem 7(4), Fig. 7), so the tokenizer’s split is one path per block and only the last block can change when more bytes follow. The engine commits the whole string when no allowed next byte can merge into its last block, and otherwise holds the last block back and requires the next token to cover it. This is a quality matter, not a speed matter: without it, a model given the forced bytes `"` `:` can pick the token `"To`, which in training almost always begins `"Today`, and an extraction drifts to `"To Be Determined` instead of `"Maria Gonzalez`. On the first workloads the rule raised the share of the unconstrained answer’s values that the constrained output reproduces from 0.76 to 0.82 on a 1.5B model and from 0.43 to 0.57 on a 3B one.

C.4 No table at all: the walk kernel

The table of Section C.1 is one side of a trade between time and space: (vocabulary $\times$ states) entries, computed once, 77 MB to 1.5 GB. The proof behind the ReachUL membership is the other side: a configuration is reached by one path, so nothing about it needs to be stored, it can be recomputed from the path whenever it is needed. For the step graph this recomputation is easy to parallelize, because the paths *are* the tokens. Each decode step, one GPU kernel runs a thread per vocabulary token: the thread walks its token’s bytes through the byte automaton from the current state (a handful of lookups, since a token has a handful of bytes) and writes the state it reaches, or DEAD. That row is the step’s mask and its transition, exactly what the table’s column would have been, the tests compare the two entry by entry. The state stays on the device, so the pipelined loop is unchanged, and the memory is the automaton, 512 bytes per state, plus the vocabulary’s bytes once: about 3 MB for every grammar in this paper. The kernel costs 7–18 microseconds per step against a decode step of 7–15

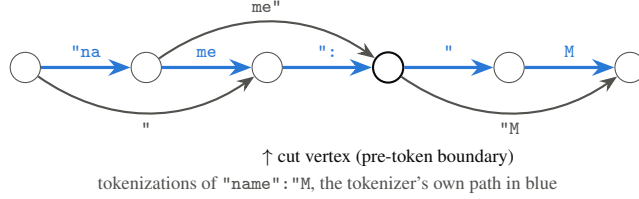


Figure 7: **The tokenization lattice.** Each path from left to right is one way to split the forced string into tokens. Tokens never cross a pre-token boundary, so the marked vertex lies on every path: it is a cut vertex, and the tokenizer’s split is one choice per block (Theorem 6). Only the block after the last cut vertex can change when more bytes arrive, which is what the boundary rule guards.

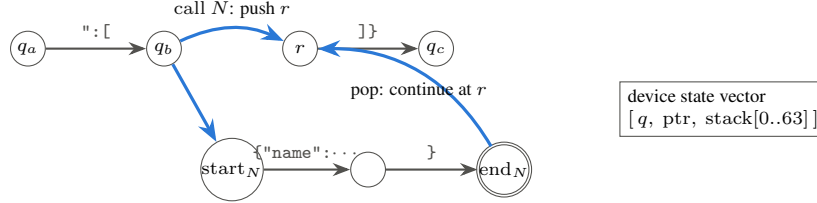


Figure 8: **A call and its return.** At q_b the only continuation is a child node, an occurrence of the nonterminal N : the machine pushes the return state r and continues at N ’s start; N ’s closing bracket pops and continues at r . The whole configuration, including the stack, is one small integer vector on the GPU.

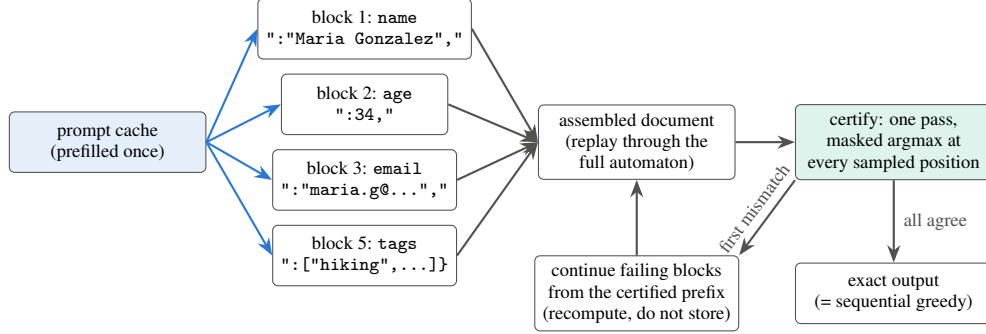
milliseconds, and there is no compile beyond the automaton. States at which few tokens are allowed keep a sparse output head: their allowed token ids are computed once on the host and kept as ids, at most 16 KB per state. The batch engine runs the same kernel over all its rows with one combined automaton table.

C.5 Recursive schemas: the pushdown

A schema whose definition refers to itself, a tree of nodes with children, an expression tree, a threaded discussion, has no regular expression. The library we build on unrolls the recursion three levels and at the deepest level drops the recursive field but keeps its comma, so it accepts $\{ "v" : 1, \}$ and nothing deeper: recursive schemas were not merely limited but broken. Our construction is the machine of Theorem 4, the second instance of Theorem 7: the control machine is a deterministic pushdown and the context is the state with the top two stack entries. Each recursive definition is its own automaton, a reference to it is a *call edge* that pushes the return state and continues at the callee’s start, a callee that reaches its closing bracket pops and continues at the return state (Fig. 8). The compile checks that the machine is deterministic and refuses grammars where it is not (an LL(1) condition at every call site). The token table is again one pass down the trie, now with rows that carry a small symbolic stack: a token that ends exactly at a return is context-free, a token that continues past a return depends on the popped return states, and those few entries (7–12 tokens and 4–11 states on the schemas below) live in a small table indexed by the top two stack entries. The stack itself is a 64-entry array in the state vector on the GPU, so decoding stays pipelined; deeper pushes are masked, which is the cap of Algorithm 1. Exactness is tested against byte-by-byte stepping on every reachable configuration of a small vocabulary and on tokenized random trees with the real one.

C.6 Decisions, not tokens: blocks in parallel

On a laptop the right unit of cost is the number of weight streams an output needs, and Theorem 7 says what that number is. A sequential decoder spends one stream per sampled token. But only the *decisions* along the path need the model: a forced chain needs no stream (Section C.2), and the independent blocks between cut vertices (Theorem 6) can be decoded in parallel, as many blocks per stream as fit in a batch, and then certified exact in one further stream. With these, the streams an



one wave: the longest block's steps, at B rows per step; one certification pass; blocks that failed continue in the next wave

Figure 9: **Certified block-parallel decoding.** The blocks of the output (the leaf values of the schema) are decoded as branches of one batch from the prompt's cache. The assembled document is certified in one pass; blocks whose draft disagrees continue from the first disagreement with the correct token. The output is token-identical to sequential decoding.

output needs are bounded by the length of its longest block plus one per certification, rather than by the length of the output.

The language of an object schema is a product over its values: every key is a forced chain, so the state after "key": is a cut vertex, and the automaton decomposes into blocks, one per leaf value (nested objects whose properties are all required are flattened into their leaves). The model's distribution over the values is *not* a product, the block product is a statement about the language only. It is used as a *draft* and made exact by a *certificate* (Fig. 9). Every block is decoded as a branch of one batch from the prompt's cache, from the automaton state at which the sequential decoder would sample its first token, with the key path as its only context. The assembled document is replayed through the full automaton with the sequential commit rules, giving the token sequence D and the state q_i before each sampled position i ; one pass over D gives the logits at every position, and the certificate holds at i if the masked argmax at q_i is D_i , the same test the sequential decoder would have made. By induction on i , the longest certified prefix is exactly what sequential greedy decoding produces. At the first failure in a block, the correct token is the argmax there, the block continues from it in the next wave, its prefix recomputed from the prompt's cache rather than stored per token (the SC^2 discipline). Two waves suffice on the workloads below.

A second use of the same idea, *finite regions*, decides an enum, a boolean or a tool name in one stream: a capped search (Lemma 2) lists every completion of the region, and all of them are scored from the same prefix in one batched pass. It is exact for what it computes and, as Section D.6 reports, not a better classifier; we include it for completeness.

C.7 The engine around the theory

The mechanisms above sit inside a decode loop that a laptop needs anyway, and that part is engineering. We state it so that the comparisons are honest, it is held fixed on both sides of every constrained-decoding comparison, and it is the whole difference in the free-decoding comparisons with stock mlx-lm.

- *Kernels.* A 4-bit matrix-vector kernel tuned per weight shape for 1 to 16 rows, so that a batch of a few streams reads the weights once; one fused Metal kernel for the whole recurrent (Gated DeltaNet) step of up to 16 tokens; fp16 arithmetic, which on M1/M2 GPUs is both faster and closer to float32 than bf16 (Section D.5), the prompt pass on MLX's dense matmul from 128 rows.
- *Pipelining.* The next step's graph is built while the current step runs, the constraint's state and mask live on the GPU, so the host is never in the loop. This is what lets a masked step cost the same as a free one, and what llguidance's host-side masking cannot do.
- *Batching.* Streams join and leave a running batch at any step; every row feeds exactly one token per step (its next forced token or the token it sampled), so a step is a uniform batch; new prompts

Table 5: **How the theory is used.** For each result, the engine mechanism that instantiates it, the property it guarantees, and the measured effect.

Result	Mechanism	Guarantee	Effect
Theorem 7 (step graph)	one traversal per constraint, the same decode step for every constraint type	the step graph of any deterministic control machine is a mangrove	regular and recursive constraints in one engine
Lemma 2 (truncation)	work budget of the table pass, automaton size cap; stack depth; wave cap	a capped search is complete or reports a violation	no partial tables; no partial stacks
Construction 3 (universal source)	all states as one row of the trie pass	one rooted pass computes all-pairs entries	0.2–0.6 s per schema
Theorem 4 (capped stack)	the pushdown constraint, stack on the GPU	deterministic; every configuration reachable once	recursive schemas at 170 tok/s (2B)
Theorem 5 (branch kernel)	forced spans	one-way-out chains are certain output	30% of tokens free; 180 vs 138 tok/s
Theorem 6 (block product)	canonical split per pre-token block	tokenizations factor over cut vertices	fidelity 0.76→0.82, 0.43→0.57
Theorem 6 + SC ²	blocks decoded in parallel, certified, prefixes recomputed	the certificate is the sequential argmax at every position	374 vs 672 streams, 18/18 exact
ReachUL traversal (one path per token)	the walk kernel: the current row recomputed each step on the device	one path per token; nothing stored	3 MB vs 824 MB at 1,106 states, same speed

are prefilled in one right-padded batch, a shared prefix is prefilled once, a prompt cache keeps recent prefills within a budget.

- *Sampling.* Greedy, or temperature with top- p and top- k under one rule shared with the stock sampler, so that sampled runs of the two engines follow the same distribution.

D Speed: all measurements, theory results and engine results

Guide: Sections D.1 and D.6 are results that follow from the theorems (forced chains, the capped stack, the block product). Sections D.2–D.5 are results of the engine’s kernels and arithmetic; they do not depend on the theorems and are secondary. Each subsection ends with its one-line summary.

Setup. One machine: an Apple M2 Pro with a 19-core GPU, 16 GB of memory and a wired GPU limit of 10.7 GB. Two models: Qwen3.5-2B (18 recurrent and 6 attention layers, 1.1 GB resident) and Qwen3.5-4B (24 recurrent and 8 attention layers, 2.26 GB resident), both in 4-bit precision, with a 248k-token vocabulary. Decoding is greedy unless stated. Separate processes on this machine vary by up to $\pm 20\%$ with memory pressure, so comparisons between our configurations are made inside one process with interleaved rounds and medians, the spread within a run is under 2%. *Stock* mlx-lm runs in its own process with its own kernels. Every number names the script that produced it, the result files ship with the paper.

D.1 One agent: a constrained stream is faster than a free one (theory: Theorems 5 and 4)

Fig. 10 and Table 6 compare output tokens per second of decode on the same prompts (bench/tps_constrained.py): six JSON schemas for the regular path (a person record, a tool call, a product, an event, a classification, a contact card), five recursive schemas for the pushdown path. Under a JSON schema STCON decodes at 180 tokens/s on the 2B where the free stream runs at 138, and at 82 against 63 on the 4B: forced spans supply about 30% of the output tokens without a weight stream, and the masks, which live on the GPU, cost nothing measurable. Ilguidance’s fast-forward feeds a similar share of tokens, but its mask is computed on the host every step, which loses the overlap of host and GPU work and lands it at the speed of free decoding. End to end (prefill plus decode) on 18 extraction jobs on the 2B, STCON takes 8.0 s against 12.3 s for Ilguidance, and 6.8 s when the prompt carries only the keys and options rather than the schema’s syntax, which the forced spans supply, the share of the unconstrained answer’s values that the constrained output reproduces is 0.93 in every mode, and all outputs are valid.

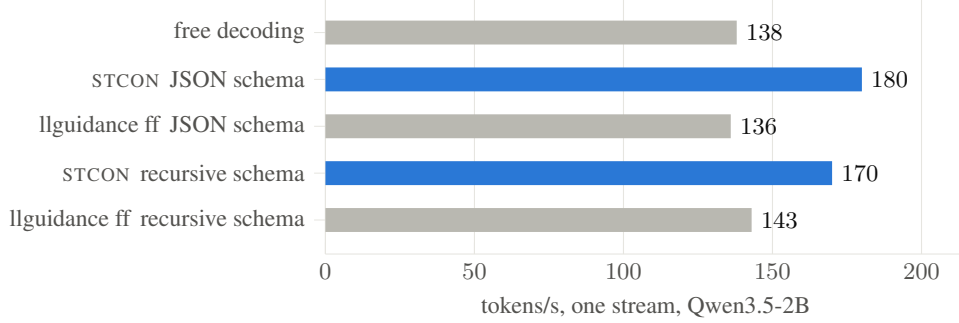


Figure 10: **Constrained decoding is faster than free decoding.** Output tokens per second of decode on the same prompts, single stream, Qwen3.5-2B, median of three interleaved rounds. The constraint is not a tax: it tells the engine what is certain, and certain text is written for free.

Table 6: **Constrained decoding, one stream** (bench/tps_constrained.py). Tokens per second of decode; weight streams per output token; share of output tokens fed as forced spans. *ff* is llguidance’s fast-forward mode.

mode	Qwen3.5-2B			Qwen3.5-4B		
	tok/s	streams/token	forced	tok/s	streams/token	forced
free decoding	138	1.01	0%	63.2	1.01	0%
JSON schema, STCON	180	0.69	31%	82.3	0.69	31%
JSON schema, llguidance ff	136	0.72	28%	68.6	0.73	27%
recursive schema, STCON	170	0.72	28%	78.6	0.70	30%
recursive schema, llguidance ff	143	0.71	29%	67.4	0.72	28%
recursive schema, llguidance	94	1.00	0%			

Recursive schemas in detail. Table 7 compares the pushdown path with llguidance, which supports recursive schemas natively, on five tasks (a Solar-System tree, an expression tree, a threaded discussion, a directory tree, a taxonomy) with three prompt phrasings each, on the 2B. On the expression task every engine fails: the model nests without end; ours stops at the depth limit, llguidance at the token limit. Per output token, ours decodes $1.6\times$ faster than llguidance and $1.19\times$ faster than its fast-forward mode, because forced spans cross returns: after a node’s last value the whole `]]`, `{"name": "` continuation is one span.

Core: Theory: forced chains supply about 30% of a JSON output without a weight stream, so a constrained stream is $1.3\times$ faster than a free one on both models and $1.2\text{--}1.6\times$ faster than llguidance; the capped stack does the same for recursive schemas.

D.2 Many agents: throughput by number of streams (engine only)

Table 8 is the first problem of Table 4 measured (bench/bench_tps.py): B streams decode 256 tokens each, and the number is the total tokens per second. Stock mlx-lm’s batch generator stalls between 4 and 8 streams (186 and 188 tokens/s on the 2B, 87 and 87 on the 4B), because its decode step reads the weights once per row; ours reads them once per step, so eight agents cost little more than one. The middle column, mlx-lm’s own loop running on our patched model, shows that the gain is the kernels and fp16, not the loop. At one stream the engines are within 5% on both models: a single stream is bound by the weight stream and no loop can change that. Nothing is shared between the streams: with the same prompt on all eight streams (`-same-prompt`) the numbers do not move (273 against 273 tokens/s for ours on the 4B, 87 against 87 for stock), and no grammar is involved.

Core: Engine only: batched kernels read the weights once per step for all rows, stock stalls between 4 and 8 streams; the same numbers with the same prompt on every stream, so nothing is shared.

Table 7: **Recursive schemas** (bench/bench_recursive.py). Five tasks $\times$ three phrasings, Qwen3.5-2B, greedy, 400-token limit. Passes are weight streams; ms per token is over valid outputs only.

Mode	Valid	Passes	Forced tokens	ms / token
STCON pushdown, forced spans, canonical rule	11/15	1,452	921	5.9
STCON pushdown, canonical rule off	12/15	1,304	875	5.8
llguidance	12/15	2,204	0	9.4
llguidance fast-forward	12/15	1,309	895	7.0

Table 8: **Free decoding, total tokens/s by number of streams** (bench/bench_tps.py, 256 tokens per stream, no constraint). *Stock* is mlx-lm in its own process; *mlx-lm on ours* is mlx-lm’s batch generator running on our patched model in the same process; *per stream* is ours divided by B .

streams	Qwen3.5-2B					Qwen3.5-4B			
	stock	mlx-lm on ours	STCON	vs stock	per stream	stock	STCON	vs stock	per stream
1	127	140	142	1.12 $\times$	142	61	64	1.04 $\times$	64
2	172	251	259	1.51 $\times$	130	82	116	1.41 $\times$	58
4	186	341	339	1.82 $\times$	85	87	160	1.83 $\times$	40
8	188	575	576	3.06 $\times$	72	87	271	3.13 $\times$	34
16	342	751	758	2.22 $\times$	47	157	341	2.17 $\times$	21
32	624	844	873	1.40 $\times$	27				
64	615	837	952	1.55 $\times$	15				
128	613	830	892	1.45 $\times$	7				

D.3 Agents end to end, and typed tool calls (engine, plus the walk kernel’s typed grammar)

The engine is packaged as an OpenAI-compatible server, and a coding agent (OpenCode) runs on it. Table 9 runs sixteen agents at once on a built-in task suite (stcon-agents, 2B, greedy), each agent making tool calls through the server, against the same agents on stock mlx-lm with llguidance for the tool-call grammar: 2.55 $\times$ less wall time and 2.16 $\times$ the throughput, with more agents finishing within the time limit. Tool calls are a grammar problem in their own right: an untyped grammar lets the model name a tool and then write any JSON as arguments, a *typed* grammar fixes every tool’s parameter names and types, which on Qwen’s vocabulary is an automaton of 1,106 states whose table (824 MB, minutes to build) did not fit in the server. With the walk kernel it costs 3.4 MB. On eight concurrent agent requests (bench/bench_typed_server.py) the 2B produces 46 of 46 calls with every argument present and of its declared type at 366–403 tokens/s, against 43 of 52 with the untyped grammar at the same speed, the 4B produces 8 of 8 typed calls either way (131 against 139 tokens/s), so on the larger model the typed grammar turns a habit into a guarantee at a cost of 6%.

Core: Engine result with one theory ingredient: the typed tool grammar exists only because the walk kernel makes its 824 MB table unnecessary; the 2.5 $\times$ wall time is the batched kernels.

D.4 LiveCodeBench (engine only)

The tool’s purpose is code, so the 4B was run on LiveCodeBench [9] (code_generation_lite, release v6, 1,055 problems with hidden tests), with the harness bench/bench_lcb.py: the 40 most recent easy and medium problems (March–April 2025; 20 LeetCode problems with starter code, 20 AtCoder problems with stdin), the official prompt, the last code block extracted, a problem passed only when every public and hidden test passes. Four engines see the same prompts and the same weights: our single-stream engine, plain mlx_lm.stream_generate, and both batched engines with eight slots and all 40 problems submitted at once (Table 10).

Two things stand out. The throughput gap is the batched one, exactly as in Table 8: at either token cap our batch engine finishes the set in 2.2–2.5 $\times$ less wall time than mlx-lm’s batch generator (196 against 79 tokens/s in total at the 4,096-token cap), while single streams are within 5%. And greedy paths on this model are fragile. With thinking off, the 4B reasons in prose at length before its code; half the answers still hit a 4,096-token cap, and the outputs of any two engines are identical on only

Table 9: **Sixteen agents at once** (stcon-agents -engine both -agents 16, built-in suite, Qwen3.5-2B, greedy). Stock is mlx-lm with llguidance for the tool-call grammar.

	stock (mlx-lm + llguidance)	STCON
wall time	74.6 s	29.3 s (2.55×)
throughput	125.5 tok/s	271.1 tok/s (2.16×)
agents finished within the limit	9/16	12/16

Table 10: **LiveCodeBench on Qwen3.5-4B** (bench/bench_lcb.py): 40 recent easy+medium problems, same prompts and weights on every engine; pass@1 over the subset, tokens/s of decode (one stream) or in total (batch), wall time for all problems. [†]Thinking on, 20 problems, 6 slots, temperature 0.6 / top- p 0.95 / top- k 20 (greedy loops in thinking mode); on our engine 5 of 20 finish within the cap and all 5 pass.

Cap	Engine	pass@1	tokens	tokens/s	wall	cut at the cap
1,024, greedy	STCON, one stream	8/40	28.2k	63.4	481 s	23
	mlx-lm, one stream	11/40	27.6k	62.3	499 s	22
	STCON, batch 8	9/40	27.4k	160.4	171 s	22
	mlx-lm, batch 8	10/40	28.3k	74.5	379 s	23
4,096, greedy	STCON, one stream	9/40	96.0k	59.1	1,667 s	22
	mlx-lm, one stream	14/40	87.2k	61.1	1,482 s	17
	STCON, batch 8	12/40	93.3k	196.1	476 s	20
	mlx-lm, batch 8	13/40	93.0k	78.7	1,181 s	19
16,384, thinking [†]	STCON, batch 6	6/20	275k	119.8	2,296 s	15
	mlx-lm, batch 6	1/20	4.4k	—	out of memory	—

2–6 of 40 problems; mlx-lm’s own single-stream and batched paths agree on 4 of 40, five of them parting at the very first token, a near-tie between opening with prose and opening with the code block. So the pass rates differ by a few problems in either direction between engines that compute the same model. Table 11 asks, at every such divergence, which of the two competing tokens the float32 model prefers: against our engine it prefers ours on 32 of 35 (median gap 0.06 logits), against mlx-lm’s batched path it prefers mlx-lm’s single path on 22 of 36, with gaps up to 1.8. The engine closer to the exact model is ours, the higher pass count of stock’s path on this subset is the luck of its rounding.

Thinking mode. With thinking on and greedy decoding the 4B loops: 16 of 20 problems ran to a 16,384-token cap inside the think block, 11 of them repeating one sentence (“Wait, if . . .”) to the end, while all 4 that finished passed. This is what Qwen’s model card warns of, its settings for code in thinking mode are temperature 0.6, top- p 0.95, top- k 20, with an output budget of 32k tokens and 81,920 for competition programming. With those settings 5 of 20 finish within 16k tokens on our engine and all 5 pass, the rest are still reasoning at the cap. mlx-lm’s batch generator on the same run raised an out-of-memory error at six 16k-token streams after one finished answer, where our engine completed the run inside the wired limit (Section E.3 explains why). For a laptop the conclusion is plain: thinking mode is precise when it finishes and needs budgets the machine cannot afford at 20–30 tokens/s per stream, the practical setting is thinking off for agent edits, where the 4B also types its tool calls by itself.

Core: Engine only: the batched kernels give 2.2–2.5× less wall time; pass rates differ by rounding at near-ties, where float32 sides with our engine; thinking mode needs budgets a laptop cannot afford.

D.5 Fidelity of the arithmetic (engine only)

Our engine computes in fp16 and mlx-lm in bf16, because Apple’s M1 and M2 GPUs have no native bf16 arithmetic. Table 12 compares both with a float32 run of the same quantized weights on 576 teacher-forced positions (bench/fp16_fidelity.py, 4B): fp16 agrees with float32 on the top token everywhere and is about 50 times closer in divergence. On the 2B, greedy generations of the two engines diverge only at near-ties, where the two competing tokens are within 0.08 logits

Table 11: **Which side is the model on?** (bench/lcb_divergence.py.) Every engine is compared with mlx-lm’s single stream. For every problem where the two outputs part, the prompt plus the common prefix is run through the model in float32 and the logit gap between the two competing tokens is read (a near-tie is a gap of a few tenths of a logit or less), the last column counts which token float32 prefers.

cap	engine	divergent	median gap	max gap	float32: mlx-lm	float32: other
4,096	STCON, one stream, fp16	35	0.056	0.52	3	32
	STCON, batch of 8, fp16	34	0.062	0.52	2	32
	mlx-lm, batch of 8	36	0.145	1.83	22	14
1,024	STCON, one stream, bf16	38	0.097	0.33	23	15

Table 12: **Compute dtype against float32** (bench/fp16_fidelity.py, Qwen3.5-4B, six prompts, 96 positions each).

dtype	top-1 agreement	mean KL	p99 KL	max KL	max $ \Delta \text{logit} $
bfloat16 (mlx-lm)	0.9965	1.67×10^{-4}	2.0×10^{-3}	3.0×10^{-3}	5.60
float16 (STCON)	1.0000	3.1×10^{-6}	4.6×10^{-5}	9.1×10^{-5}	0.31

in float32, the same study on twelve generic prompts of the 4B gives 5 of 12 identical 200-token generations, all gaps within ± 0.27 , float32 siding with ours on 5 of 7 in bf16 and 6 of 7 in fp16.

Core: Engine only: fp16 is closer to float32 than bf16 on this GPU, so our engine’s greedy path is the one the exact model prefers.

D.6 Decisions, not tokens: blocks in parallel (theory: Theorem 6)

Table 13 compares sequential decoding (the same commit rules) with certified block-parallel decoding on the six schemas and three prompt variants each (bench/bench_blocks.py). Over the 18 outputs the sequential decoder needs 672 weight streams on the 2B and the block decoder 374 ($1.80\times$ fewer); on the 4B, 686 against 484 ($1.42\times$). All 36 outputs are token-identical to the sequential ones. Wall time is not yet lower on this machine, 7.4 against 5.9 s on the 2B and 18.6 against 12.2 s on the 4B, because a step over 5–9 rows costs $1.7\text{--}2.4\times$ a single-row step on the M2 Pro, whose GPU is compute-bound from four rows, and a certification pass over 40 tokens about five. The count of weight streams is the hardware-independent number; where extra rows of a batch are nearly free, wall time follows it. Finite regions, the other mechanism of Section C.6, decide a label among 16 prefix-sharing categories in 2.0 streams against 2.9 for token-level decoding, but agree with the intended label less often (7 against 15 of 24), because the most probable complete string is the short generic label; we report it as a negative result for sequence-level decisions on such labels.

Core: Theory: the block product says which parts of an output are independent and the certificate makes the parallel draft exact; $1.4\text{--}1.8\times$ fewer weight streams, identical output, no wall-time gain yet on this GPU.

E Memory: all measurements, theory results and engine results

Guide: Sections E.1 and E.2 follow from the theorems (the walk kernel is the ReachUL proof’s discipline of recomputing rather than storing). Sections E.3 and E.4 are engine results (the prefill transient, cache growth, and the budget they imply); they do not depend on the theorems and are secondary. Each subsection ends with its one-line summary.

Memory is measured with MLX’s own counters (active and peak bytes on the GPU) inside each engine’s process, the process’s resident size is meaningless for unified memory and is not used. Everything below is on the 4B unless stated, the 2B numbers are given where they differ in kind.

Table 13: **Weight streams per output** (bench/bench_blocks.py): six schemas, three prompt variants each, greedy; sequential against certified block-parallel decoding, which produces the same tokens in every case.

schema	Qwen3.5-2B			Qwen3.5-4B		
	sequential	blocks	exact	sequential	blocks	exact
person	84	33	3/3	84	33	3/3
tool_call	35	17	3/3	35	20	3/3
product	112	53	3/3	112	46	3/3
event	169	96	3/3	167	109	3/3
classification	126	112	3/3	142	159	3/3
contact_card	146	63	3/3	146	117	3/3
all 18 outputs	672	374	18/18	686	484	18/18
decode wall	5.9 s	7.4 s		12.2 s	18.6 s	

Table 14: **Constraint memory against grammar size** (bench/bench_walk.py). 248k-token vocabulary, pipelined engine, greedy, no healing on either side; tok/s median of three interleaved rounds. The table’s memory is $3|Q|V$ bytes, the walk’s is the automaton plus the packed vocabulary. Outputs are identical wherever the table exists.

grammar	states	table	build	walk	kernel	tok/s, 2B		tok/s, 4B	
						table	walk	table	walk
person	104	77 MB	0.7 s	2.9 MB	12–14 μ s	156	153	75	75
tool_call	164	122 MB	1.0 s	2.9 MB	7–9 μ s	263	256	124	121
product	192	143 MB	1.3 s	2.9 MB	8–9 μ s	191	189	90	89
event	178	133 MB	1.3 s	2.9 MB	7–9 μ s	168	161	80	78
classification	158	118 MB	1.0 s	2.9 MB	7–9 μ s	159	169	82	83
contact_card	246	183 MB	1.7 s	3.0 MB	7–8 μ s	183	180	88	87
48 fields	986	735 MB	9 s	3.3 MB	9–14 μ s	274	267	131	132
typed tool calls	1,106	824 MB	62–127 s	3.4 MB	11–18 μ s	206	189	89	87
96 fields	1,970	1.5 GB	not built	3.85 MB	8–14 μ s	—	260	—	121

E.1 Per grammar: the table against the walk (theory: the ReachUL walk kernel)

Table 14 is the second problem of Table 4 measured (bench/bench_walk.py), on grammars from 104 to 1,970 states: the six schemas, two synthetic schemas with 48 and 96 fields, and the typed tool-call grammar. A table is 3 bytes per (state, token), so it grows with the states from 77 MB to 1.5 GB, and its construction from 0.7 s to a minute, the walk kernel needs 2.9–3.85 MB for every grammar, 7–18 microseconds per step, and decodes at the table’s speed with identical tokens on every grammar where the table could be built. What remains of the compile is the automaton itself: 0.02–0.5 s for the schemas and 27 s for the tool grammar, whose subset construction, not its table, is the cost. On the 4B the same process that had built the tables of the six schemas next to the 3 GB model no longer had room for the typed grammar’s 824 MB table: it took 332 s to build and decoded at 6 tokens/s under swap pressure, and the walk in the same pressured process ran at 25 tokens/s, the row in the table is from a fresh process. On the 2B the same process fit. The typed table is the one that Section D.3 could not afford in the server.

Core: Theory: the walk kernel replaces a 77 MB–1.5 GB table by 3 MB at the same speed with identical output, because every token has one path and the row can be recomputed each step.

E.2 In the wild: a server that sees many grammars (theory: the ReachUL walk kernel)

Table 14 is one grammar at a time. Fig. 4 and Table 15 are the third problem of Table 4: one server process is fed sixteen distinct grammars in turn (the six schemas, then schemas of 8 to 96 fields), four requests each, three concurrent, with the prompt cache held at 50 MB so that what grows is the constraints, and its GPU memory above the model is read after every grammar (bench/bench_memory_wild.py). With token tables on the 4B the memory reaches 1.5 GB after

Table 15: **The same server, both models** (bench/bench_memory_wild.py): grammars served before failure, GPU memory above the model after the last grammar served, and the longest first-request wait.

model, prompt cache	constraints	served	above the model	longest first request
2B, 50 MB	token tables	15/16, then OOM	8.2 GB	26 s
2B, 50 MB	walk kernel	16/16	0.6 GB	2.8 s
2B, 1.5 GB	token tables	14/16, then OOM	7.6 GB	
2B, 1.5 GB	walk kernel	16/16	2.2 GB	
4B, 50 MB	token tables	14/16, then OOM	6.9 GB	26 s
4B, 50 MB	walk kernel	16/16	1.1 GB	6.6 s

Table 16: **Peak memory by decoding regime, Qwen3.5-4B** (bench/bench_memory_4b.py): B streams $\times N$ tokens on LiveCodeBench prompts, no constraint, EOS ignored. *Above the model* is the peak minus the 2.26 GB of resident weights. The out-of-memory entry is the thinking run of Table 10.

streams $\times$ tokens	STCON		stock mlx-lm	
	above the model	tok/s	above the model	tok/s
1 $\times$ 4096	1.0 GB	65	0.8 GB	60
8 $\times$ 64 (prefill only)	6.4 GB		3.8 GB	
8 $\times$ 1024	6.4 GB	175	3.7 GB	82
8 $\times$ 4096	6.4 GB	212	5.2 GB	84
6 $\times$ 8192	4.9 GB	93	5.9 GB	64
6 $\times$ 16384	4.9 GB	128	out of memory	—

the six schemas, 5.5 GB after thirteen and 6.9 GB after fourteen, when the fourteenth grammar’s first request waits 26 s for its table and throughput collapses to 11 tokens/s under the pressure, the fifteenth grammar kills the process with a Metal out-of-memory error at 11.1 GB active. With the walk kernel all sixteen are served within 1.1 GB, and a new grammar’s first request waits 1.2–6.6 s, for its automaton. On the 2B the shape is the same one model size lower: tables reach 8.2 GB after fifteen grammars and the sixteenth fails, the walk serves all sixteen within 0.6 GB; with the prompt cache at its usual 1.5 GB, 7.6 GB and a failure at the fifteenth against 2.2 GB for all sixteen.

Core: Theory: with tables a server dies at the fifteenth or sixteenth grammar; with the walk kernel it serves all sixteen in 0.6–1.1 GB.

E.3 Decoding regimes: where the peaks come from (engine only)

Table 16 measures the last problem of Table 4, and one of our own (bench/bench_memory_4b.py): B streams decode N tokens each on LiveCodeBench prompts (560 tokens on average, 1,360 at most), and the peak GPU memory above the 2.26 GB model is read at the end. Two different things set these peaks, and Table 17 separates them (bench/probe_prefill_memory.py).

The prefill transient. At short contexts the peak is the prompt pass, not the caches: our 8 $\times$ 64 and 8 $\times$ 4096 peaks are the same 6.4 GB. Our batch engine prefills every admitted prompt in one right-padded batch with 1,024-token chunks, and that costs about 0.8 MB per token in the pass (rows $\times$ chunk): 8 rows at chunk 1,024 are 6.5 GB, at chunk 256 they are 2.4 GB, and single-stream prefill gets 11% slower. Stock prefills four prompts at a time in 512-token steps but computes the logits at every prompt position, about 1 MB per token, hence its 3.7 GB. In our single-stream engine the prompt pass applies the whole model including the output head, so a 1,360-token prompt costs a nearly constant 1.1–1.4 GB whatever the chunk. Both are knobs, not costs of the method: capping a prefill pass at about 2,000 tokens (rows $\times$ chunk) brings the 8-slot peak to about 2.4 GB, and skipping the head in the single-stream prompt pass saves about 1.2 GB. We report them because they are what a reader who deploys the engine with 16 slots needs to know: sixteen 1,000-token prompts admitted at once would need 13 GB today.

Cache growth. Our caches never exceed the prefill transient in these runs: the peak at 6 $\times$ 8192 equals the peak at 6 $\times$ 16384, although the caches at 16k tokens hold 3.3 GB (32 KB per token per

Table 17: **The prefill transient** (bench/probe_prefill_memory.py, Qwen3.5-4B, our engine, nothing decoded): peak memory above the model while prefilling the longest prompt (1,360 tokens) alone, and B prompts (1,360 down to 703 tokens) in one right-padded batch, by prefill chunk.

	chunk (tokens per pass)				
	2048	1024	512	256	128
one stream, 1,360 tokens	1.41 GB	1.46 GB	1.23 GB	1.09 GB	1.19 GB
batch of 2		2.12 GB		1.36 GB	
batch of 4		3.50 GB		1.59 GB	
batch of 8		6.48 GB		2.43 GB	

stream, plus 50 MB of recurrent state per stream). Stock grows its KV cache by copying, so growth costs about twice the raw KV (its peak rises by 1.5 GB from 1,024 to 4,096 tokens for 0.75 GB of cache), at 8,192 tokens it passes ours, and at 16,384 it exceeds the wired limit, which is what killed the thinking run in Table 10.

Core: Engine only: at short contexts the peak is our batched prefill, a knob; at long contexts stock’s cache growth exceeds ours and runs out at six 16k streams.

E.4 A memory budget for n agents (engine only)

The numbers above fit one formula. Let W be the resident weights, C the prompt-cache budget, P the prefill transient, n the streams, T the tokens each holds, and G the constraints. Then the peak GPU memory is about

$$M \approx W + C + P + n(s + \tau T) + G,$$

with, on Qwen3.5-4B, $W = 2.26$ GB, $s = 50$ MB of recurrent state per stream, $\tau = 32$ KB per token per stream, $P \approx 0.8$ MB $\times$ (rows $\times$ chunk) for our batch prefill (or the capped value), and G equal to 3 MB per grammar with the walk kernel against $3|Q|V$ bytes, 77 MB to 1.5 GB, per grammar with tables, never freed. For stock mlx-lm, $P \approx 3.7$ GB and the cache term is doubled by growth. The machine has 10.7 GB.

Eight coding agents with 1,000-token prompts, 4,000-token answers and four schemas between them need, on our engine today, $2.26 + 1.5 + 6.4 + 8 \times (0.05 + 0.13) + 0.01 \approx 11.6$ GB: too much, and entirely because of the prefill batch; with the 2,000-token prefill cap the same agents need about 7.6 GB and fit, and sixteen agents with the same prompts fit at about 9 GB. On stock mlx-lm the same eight agents need $2.26 + 1.5 + 3.7 + 2 \times 1.44 + 0.5 \approx 10.8$ GB with tables for the four schemas, at the edge of the limit, and any longer answer pushes them over, the thinking run of Table 10 is that case. The two terms that decide how many agents a laptop holds are the prefill transient and the constraints, and both are set by the engine, not by the model.

Core: Engine only: the budget formula; the prefill transient and the constraints decide how many agents fit, and both are set by the engine, not the model.

F Exactness and tests

None of the mechanisms changes what greedy decoding produces, except the canonical split, which changes it on purpose. Certified block-parallel decoding is token-identical to sequential decoding on all 36 outputs of both models, the walk kernel’s rows equal the table’s columns for every state, and its outputs are identical on every grammar, single-stream and batched; lossless forced spans give byte-identical output to plain constrained decoding with fewer passes, the recursive path’s tables equal byte-by-byte stepping on every reachable configuration of a small vocabulary and on tokenized random trees with the real one, the sampling rule is tested for top- k , top- p and masks. The test suite (234 tests) runs on both models.